VITyarthi | Open Source Software

The Open Source Audit

A Capstone Project for OSS NGMC Course

Course: Open Source Software | Unit Coverage: 1 – 5
Max Marks: 100

Student Name	Registration Number
Nitin Chauhan	24BCE10838
Slot	**Date of Submission**
B22	28/03/2026

1. Introduction

Most of the software that powers the modern world was not built by a single company and sold for profit; it was built openly, shared freely, and improved collectively . This project is an audit of one such foundational tool: **Git**.

This report will not merely explain Git's technical commands, but will explore its origin, the philosophy of its creator, the specific freedoms guaranteed by its license, and its broader impact on the global software ecosystem. Alongside this theoretical analysis, practical Linux footprint mapping and shell scripting tasks will demonstrate how automation and command-line mastery connect directly to the open-source ethos of transparency . Understanding these foundations is not an optional academic exercise; it is a prerequisite for modern professional software development.

Why this matters	Every line of code you write in your career will sit on top of open-source foundations. Understanding the philosophy behind that is not optional — it is part of being a professional.

2. Choosing Your Software

Approved software options

Software	Category	License	Why it's interesting
Linux Kernel	Operating System	`GPL v2`	*The foundation everything else runs on*
Apache HTTP Server	Web Server	`Apache 2.0`	*Powers ~30% of all websites globally*
MySQL	Database	`GPL v2 / Commercial`	*A license with a dual-model story to tell*
Firefox	Web Browser	`MPL 2.0`	*A nonprofit fighting for an open web*
VLC Media Player	Multimedia	`LGPL / GPL`	*Plays anything — built by students in Paris*
LibreOffice	Office Suite	`MPL 2.0`	*Born from a community fork — a real lesson*
Python	Programming Language	`PSF License`	*A language shaped entirely by community*
Git	Version Control	`GPL v2`	*The tool Linus built when proprietary failed him*

Your choice	**Git**

3. Project Report

Part A — Origin and Philosophy (Units 1 & 2)

A1. The problem this software was created to solve

Git was created in April 2005 by Linus Torvalds, the creator of Linux, to manage the development of the Linux kernel.

Why Git Was Created (Origin Story)

The creation of Git was triggered by a crisis in the Linux kernel community:

1. **Licensing Dispute:** Since 2002, the Linux kernel project had used a proprietary version control system called BitKeeper. In 2005, the relationship between the Linux community and BitKeeper's parent company broke down, and the tool's free-of-charge status for open-source projects was revoked.

2. **Lack of Alternatives:** Torvalds found that no existing free version control systems (like CVS or SVN) met the Linux project's extreme needs for speed, scale, and distributed workflow.

3. **Design Goals:** Torvalds wanted a system that could handle thousands of parallel branches, protect against data corruption, and execute tasks like patching in seconds rather than minutes.

How Git Was Created:

Torvalds began developing Git as a "content-addressable filesystem" rather than a traditional source control manager**.**

1. **Rapid Initial Development:** Torvalds wrote the initial framework in a little over a week, starting on April 3, 2005.

2. **Self-Hosting**: By April 7, just four days after the first line of code was written, the project became "self-hosting," meaning Git was being used to track its own source code.

3. **Benchmark Success**: Within a month, Git was benchmarked recording patches to the Linux kernel at a rate of 6.7 patches per second, meeting its ambitious speed goals.
4. **Handover**: In July 2005, Torvalds turned over the maintenance of the project to Junio Hamano, who has been the primary maintainer ever since and led it to its 1.0 release in December 2005.

Git emerged from need and exasperation. In the early 2000s, the Linux kernel community was large and dispersed, using a proprietary version control system named BitKeeper to handle numerous contributions. In 2005, the connection between the community and BitMover (the firm behind BitKeeper) collapsed, leading to the cancellation of the free license for Linux developers.

Linus Torvalds required a substitute, yet no current system satisfied his requirements: it had to be entirely distributed, extremely fast, and able to manage intricate, non-linear development (branching and merging). Since there was nothing, Linus created Git. He opted to disclose it freely because the Linux project depended on collaborative openness; a proprietary tool had already shown to be a single point of failure

A2. The license — what it actually says

The four freedoms of free software, as defined by the Free Software Foundation (FSF), are essential rights that guarantee users control over their computing.

The Four Freedoms:

Freedom 0 (Run): The freedom to run the program as you wish, for any purpose.

Freedom 1 (Study/Modify): The freedom to study how the program works and change it to do what you wish. Access to source code is a precondition.

Freedom 2 (Share): The freedom to redistribute copies to help others.

Freedom 3 (Improve/Distribute): The freedom to distribute copies of your modified versions to others.

Does Git Software's License Grant All Four?

Yes, Git software is released under the GNU General Public License version 2.0 (GPLv2), which is a strong copyleft, free software license that grants all four freedoms.

Freedom 0: Git can be run for any purpose, including commercial use.

Freedom 1 & 3: The source code of Git is freely available and modifiable.

Freedom 2: Users can freely redistribute exact copies of Git.

Copyleft Protection: The GPLv2 license ensures that modifications to Git must be released under the same license, ensuring the software remains free for all users.

- Generally, a company cannot modify Git and sell it without sharing their changes. Git is licensed under the GNU General Public License v2 (GPLv2). This "copyleft" license requires that if you modify and distribute (sell/transfer) the software, you must make the modified source code available to the recipients under the same GPLv2 license.

Why They Cannot (Legally):

GPLv2 Requirements: The GPL license specifically mandates that any derivative works distributed commercially must also be released under the GPL. If a company sells a modified version of Git, they must provide the modified source code to customers.

Copyright Compliance: While they can sell the software for profit, they cannot change the license or hide the modifications made to the original GPL-covered code.

The Exceptions:

Internal Use: A company can modify Git, use it internally, and keep those changes private indefinitely. The obligation to share source code is only triggered upon distribution (selling or giving the software to someone else).

Components: If they only create a separate, proprietary, and independent plugin for Git that does not link to or combine with the core GPL code, they may not need to open-source the plugin. However, modifying the core Git code itself requires adhering to the GPL license.

The primary difference is that *GPL* is a restrictive "copyleft" license requiring derivative works to remain open-source, *while MIT* is a permissive license allowing free use, modification, and proprietary distribution. In Git, GPL ensures your code’s offspring stay open; MIT allows others to make your code proprietary.

Which to choose?

- **Choose MIT:** If you want maximum adoption, allowing companies to use, change, and sell your code in proprietary products (e.g., in a private Git repo). It is simple and favored for libraries.
- **Choose GPL:** If you want to force all improvements and derivative works to be shared back with the community, ensuring the code remains free forever.

Why: I would choose **MIT** for most personal projects on GitHub to encourage widespread adoption and contribution without legal constraints, unless the goal is specifically to mandate open-source sharing of future improvements.

- **Yes**, there is a fundamental difference between "free as in free beer" and "free as in freedom" (often called "free as in speech").
- **Free as in Free Beer (Gratis):** Refers to zero monetary cost. You do not pay money to use the software, but you may not have the right to modify it, see its source code, or redistribute it.
- **Free as in Freedom (Libre):** Refers to the liberty to run, study, change, and distribute the software. It is a matter of rights, not price. Free software can sometimes cost money to acquire.

Key Differences

Feature	Free as in Beer (Gratis)	Free as in Freedom (Libre)
Focus	Price (zero cost)	Rights/Liberty
Source Code	Usually Hidden	Accessible
Modification	Restricted	Allowed
Redistribution	Restricted	Allowed
Example	Adobe Reader, Skype	Linux, Firefox, Git

Example for Git

Git is a "free and open source" software shared under the GNU General Public License v2.0, meaning it embraces both concepts:

1. **Git as in Free Beer:** You can download, install, and use Git on your machine without paying any licensing fees.
2. **Git as in Freedom:**
   1. Study/Modify: You can download the source code of Git, change how it works, and recompile it.
   2. Redistribute: You can share your modified version of Git with others.
   3. Commercial Use: You can use Git for commercial projects without restriction.
   4. Example: A company can create a custom, proprietary tool that uses Git's core functionality, or a developer can fork the Git source code to create a specialized branch to improve its speed on specific file types.

A3. The ethics of open source

Open source is not just a licensing model — it is a statement about how knowledge and tools should be shared.

Whether all software should be open source is a debate between promoting transparency and collaborative innovation versus protecting intellectual property and enabling monetization. While open source offers security, customization, and community collaboration, proprietary software provides dedicated support, user-friendly interfaces, and financial incentives for innovation.

Arguments For Open Source Software (OSS):

- **Transparency and Security:** Open code allows anyone to inspect it, making it easier to find bugs and vulnerabilities, which can lead to better security.
- **Innovation and Collaboration:** Developers can build on existing work, speeding up innovation and reducing redundant work.
- **Freedom from Vendor Lock-in:** Users are not dependent on a single company for updates or support.

- **Customization:** Users can modify the software to meet their specific needs.

Arguments Against All Software Being Open Source:

- **Funding and Sustainability:** Developing high-quality software is expensive, and proprietary models allow developers to make a profit from their work.
- **Ease of Use:** Open source projects often lack the user-friendly design and polish found in commercial, proprietary applications.
- **Support and Maintenance:** OSS may lack dedicated, 24/7 technical support or professional documentation found in commercial software.
- **Security Through Obscurity (Partial):** While not foolproof, some argue that closed code prevents bad actors from easily finding vulnerabilities.
- **Intellectual Property Protection:** Companies may not want to reveal their proprietary algorithms.

- **Conclusion:**
  While open-source software provides significant advantages in collaboration and transparency, a "one-size-fits-all" approach may not be feasible. A hybrid ecosystem where critical infrastructure is open-source, while specialized tools can be proprietary, often provides the best balance.

Whether it is ethical for a company to build a profitable product entirely on free community labor is a complex, debated topic in the software industry, often balancing legal compliance with moral reciprocity.

While many argue that the core purpose of open-source software (OSS) is to be freely used, modified, and monetized, critics argue that when profitable companies fail to contribute back to the projects they depend on, it becomes a form of exploitation.

Here is an analysis of the ethical, practical, and legal perspectives based on the examples of companies like Red Hat, Google, and Meta.

1. Arguments that it IS Ethical

- **The Intent of Open Source:** OSS is designed to be free, not just in price but in liberty. Open-source licenses (like MIT, Apache, or BSD) are created to allow unrestricted use, including commercial, without requiring contributions back.

- **"Free" as a Gift:** Many maintainers create and release software as a hobby or gift to the world, operating on the expectation that they may not receive financial return.
- **Value Addition (Not Stealing):** Companies often provide value beyond the base code, such as enterprise support, security hardening, user-friendly interfaces, or cloud infrastructure. Red Hat's model, for example, is recognized as building a business through support and service, not just by "stealing" code.
- **Bug Fixes and Stability:** The sheer scale of adoption by companies like Google and Meta allows them to test and improve software at a scale no small volunteer team could, often feeding fixes back into the community.

2. Arguments that it is UNETHICAL

- **The "Tragedy of the Commons":** When large companies extract massive value from free code but fail to support the developers, projects can become unsustainable, leading to burnout and security vulnerabilities, as seen in the Log4j issue and the XZ Utils crisis.
- **Exploitation of Labor:** It is often argued that it is unethical to use workers as a ladder for profit without sharing benefits, particularly when the company acts as a profit-focused organization while relying on non-profit labor.
- **"Open Core" and Bait-and-Switch:** Some companies use open source as a lead-generation tool, engaging with the community to build a product, and then changing licenses or closing source code once the product becomes successful.

3. Case Studies: Red Hat, Google, Meta

- The companies mentioned follow different models:
  - Red Hat (RHEL)**:** Known for being a massive contributor to the Linux kernel and associated projects. Their model relies on charging for support, certification, and stability, rather than selling the code itself.
  - Google and Meta**:** These companies contribute heavily to the projects they rely on (e.g., Linux, Kubernetes, PyTorch) because doing so ensures the technology remains reliable and tailored to their needs. However, they are also accused of adopting open source for internal efficiency and keeping their proprietary enhancements private.

4. What constitutes Ethical Behavior?

- The prevailing view, as noted by Keitaro insights, is that companies should:
  - **Contribute back:** Actively contribute code, fixes, or documentation.
  - **Support maintainers:** Financially support the developers behind critical components (e.g., via GitHub Sponsors).

- **Respect licenses:** Abide by the license terms, particularly copyleft licenses (like GPL) that require sharing modifications.

Summary Table

Aspect	Perspective
Legal	Almost always legal if license terms are followed.
Ethical	Debatable. Considered ethical if they give back; exploitative if they only take.
Sustainability	Relies on a healthy, reciprocal ecosystem to prevent burnout.

- In conclusion, it is not inherently unethical, but it becomes so when corporations treat community labor as a free resource to be exploited, rather than a collaborative partnership.

Part B — Linux Footprint (Unit 2)

Installation

```
sudo apt install git
```

Key Directories

- Binary: `/usr/bin/git`
- Config: `/etc/gitconfig`
- User config: `~/.gitconfig`

User & Group

Runs as: current logged-in user

Why important?

- Prevents unauthorized access
- Maintains security boundaries

Service Management

Git is not a service → it's a CLI tool

Updates

```
sudo apt update && sudo apt upgrade
```

Part C — The FOSS Ecosystem (Units 3 & 4)

Dependencies

- libc
- OpenSSL
- curl

Inspired Tools

- GitHub
- GitLab
- Bitbucket

LAMP Relation

Git is used to:

- manage code for web apps
- deploy updates

Community

- Maintained by Linux Foundation
- Huge contributor base worldwide

Part D — Open Source vs Proprietary (Critical Analysis)

Dimension	Git (Open Source)	GitHub Enterprise (Proprietary Alternative)
Cost	Free	Paid
Security (who can audit the code?)	Transprent	Limited Visibilty
Support and reliability	Community	Dedicated
Freedom to modify	Full Freedom	Restricted
Community vs corporate control	Decentralized	Centralized

Verdict

Open-source solutions like Git are really good because they are transparent, flexible and do not cost a lot of money. The code is available to the public so anyone can check it for security problems, which makes it safer. Git is also decentralized, which means that no one company is in charge. It is easier to come up with new ideas and it can survive even if something goes wrong.. Because Git is free and can be changed to fit your needs it is great for people who are learning trying new things and for projects that need to grow especially for new companies or school projects.

On the hand some people like to use proprietary solutions like GitHub Enterprise because they offer help when you need it are reliable and have features that big companies need. These solutions can make things more efficient. Help people stay on track. For example they have dedicated support, control over everything and tools that work together. When you are actually using something in the world I think it is a good idea to use a combination of both. Use Git as the main system and use enterprise platforms when you need to. Yes I would definitely contribute to open-source projects like Git because they help me learn and get better and they also make the community of developers, around the world stronger.

4. Shell Script Tasks

Script 1 — System Identity Report (Unit 1 & 2)

Starter structure

```
#!/bin/bash
# Script 1: System Identity Report
# Author: Nitin Chauhan

STUDENT_NAME="Nitin Chauhan"
SOFTWARE_CHOICE="Git"

KERNEL=$(uname -r)
USER_NAME=$(whoami)
UPTIME=$(uptime -p)
DATE=$(date)
DISTRO=$(lsb_release -d | cut -f2)

echo "================================"
echo " Open Source Audit — $STUDENT_NAME"
echo "================================"
echo "Software : $SOFTWARE_CHOICE"
echo "Kernel   : $KERNEL"
echo "User     : $USER_NAME"
echo "Distro   : $DISTRO"
echo "Uptime   : $UPTIME"
echo "Date     : $DATE"
echo "License  : GPL-based Linux system"
```

Script 2 — FOSS Package Inspector (Unit 2)

Starter structure

```
#!/bin/bash
PACKAGE="git"

if dpkg -l | grep -qw $PACKAGE; then
 echo "$PACKAGE is installed."
 apt show $PACKAGE | grep -E 'Version|Maintainer|Description'
else
 echo "$PACKAGE is NOT installed."
fi

case $PACKAGE in
 git) echo "Git: distributed version control revolution" ;;
 apache2) echo "Apache: backbone of web servers" ;;
 mysql) echo "MySQL: database for modern apps" ;;
 vlc) echo "VLC: plays everything freely" ;;
 *) echo "Unknown package" ;;
esac
```

Script 3 — Disk and Permission Auditor (Unit 2)

Starter structure

```
#!/bin/bash
DIRS=("/etc" "/var/log" "/home" "/usr/bin" "/tmp")

echo "Directory Audit Report"
echo "----------------------"

for DIR in "${DIRS[@]}"; do
 if [ -d "$DIR" ]; then
 PERMS=$(ls -ld $DIR | awk '{print $1, $3, $4}')
 SIZE=$(du -sh $DIR 2>/dev/null | cut -f1)
 echo "$DIR => Permissions: $PERMS | Size: $SIZE"
 else
 echo "$DIR does not exist"
 fi
done

# Git config check
if [ -f ~/.gitconfig ]; then
 echo "Git config found:"
 ls -l ~/.gitconfig
fi
```

Script 4 — Log File Analyzer (Units 2 & 5)

Starter structure

```
#!/bin/bash

LOGFILE=$1
KEYWORD=${2:-"error"}
COUNT=0

if [ ! -f "$LOGFILE" ]; then
 echo "Error: File not found"
 exit 1
fi

while IFS= read -r LINE; do
 if echo "$LINE" | grep -iq "$KEYWORD"; then
 COUNT=$((COUNT + 1))
 fi
done < "$LOGFILE"

echo "Keyword '$KEYWORD' found $COUNT times"

grep -i "$KEYWORD" "$LOGFILE" | tail -5
```

Script 5 — The Open Source Manifesto Generator (Unit 5)

Starter structure

```


#!/bin/bash

echo "Answer three questions"

read -p "Tool you use: " TOOL
read -p "Freedom means: " FREEDOM
read -p "What will you build: " BUILD

DATE=$(date '+%d %B %Y')
OUTPUT="manifesto_$(whoami).txt"

echo "On $DATE, I believe open source is about $FREEDOM." > $OUTPUT
echo "I use $TOOL daily and aim to build $BUILD for the community." >> $OUTPUT

echo "Manifesto saved to $OUTPUT"
cat $OUTPUT


```

5. Conclusion

The study of Git as an open-source software shows us how people working together can create tools that're really good and easy to use. Git was made because it was needed. Now it is a big part of how software is made today. It helps people work together from anywhere in the world. Keeps track of changes. The GPL license means that Git is free for anyone to use, which is an idea because it means people can share what they know and what they make. When we look at how Git works with Linux we can see that it is a part of the open-source world and helps many other technologies and platforms.

If we step back and look at the picture we can see that open source is not just a way to make software it is a way of thinking that is all about being free, open and working together. Some people might disagree about how open source should be used. It is clear that it helps make new things and makes them available to everyone. When we compare Git to software that is not open source we can see that Git is very flexible and gives us a lot of control. From what we have learned Git is a tool to use in the real world and helping out with projects like this can be good, for our careers and also help make the world a better place by making the open-source community stronger.